Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Hito 3 · Examen de hito

Hito 3

El Transformer

Tokenización, atención, Transformer y mini-GPT

Examen, proyecto de fase y soluciones

Edición de octubre de 2026

Instrucciones

Este examen cierra la fase III del curso, dedicada al Transformer. Cubre los módulos 12 (tokenización y modelos secuenciales), 13 (atención), 14 (el Transformer decoder-only) y 15 (entrenamiento de un mini-GPT). Debe aprobarse antes de comenzar la fase IV.

Condiciones

Duración	2 h 30 min: unos 20 min para la parte A y 2 h para la parte B
Material permitido	calculadora científica y una hoja de fórmulas manuscrita (una cara)
Puntuación total	100 puntos: parte A, 20; parte B, 80
Aprobado	70 puntos (70 %)
Módulos	M12, M13, M14 y M15

Estructura

- **Parte A. Cuestiones** (20 puntos). Veinte preguntas de opción múltiple, de 1 punto cada una, con una sola respuesta correcta entre las cuatro propuestas. Las respuestas en blanco o incorrectas valen 0; no se penalizan los errores.
- **Parte B. Problemas** (80 puntos). Seis ejercicios de desarrollo, con la puntuación de cada apartado indicada entre paréntesis. Debe justificarse cada paso: un resultado sin razonamiento vale como máximo la mitad de su puntuación.
- **Proyecto de fase**. Se entrega aparte y se evalúa con la rúbrica del capítulo «Proyecto de fase»; no suma puntos al examen.

Convenios

- Se redondea a tres decimales salvo que se indique otra cosa. Pueden usarse $e \approx 2{,}718$, $e^2 \approx 7{,}389$ y $\ln 2 \approx 0{,}693$.
- Las matrices de pesos multiplican por la derecha ($Y = XW$) y no hay sesgos salvo que se diga lo contrario.
- 1 GB $= 10^9$ bytes. Una multiplicación seguida de una suma cuenta como 2 FLOPs.
- Los modelos con nombre inventado, como «Lince-1B», son ficticios. Los datos de GPT-2 y de LLaMA-2 son los publicados por sus autores.
- Las referencias del tipo «M13, §2.3» remiten al módulo y a la sección de los apuntes del curso.

Sobre las fuentes

Los enunciados son de redacción propia. Los ejercicios marcados con *Fuente* están adaptados de tareas y ejemplos de cursos universitarios publicados: la tarea 1 de Stanford CS336 [2], la tarea 3 de Stanford CS224N [1] y los ejercicios del libro de Jurafsky y Martin [4], texto de los cursos de procesamiento del lenguaje de Stanford. En todos los casos se han cambiado los datos para que el ejercicio pueda resolverse a mano.

Índice

Instrucciones **ii**

A Cuestiones **1**

B Problemas **5**

C Proyecto de fase **9**

A Soluciones **12**

A.1 Clave de respuestas de la parte A 12

A.2 Soluciones de la parte B 13

B Autoevaluación **25**

B.1 Cálculo de la nota 25

B.2 Puerta de salida de la fase III 25

B.3 Qué hacer si no se alcanza el umbral 26

Bibliografía **27**

Parte A

Cuestiones

Cada cuestión vale 1 punto y tiene una sola respuesta correcta. Las cuestiones A.1 a A.5 corresponden al módulo 12; A.6 a A.10, al módulo 13; A.11 a A.18, al módulo 14, y A.19 y A.20, al módulo 15.

A.1. ¿Qué hace el algoritmo BPE en cada paso de su entrenamiento? [1 p]

(a) Elimina del vocabulario el token menos probable.

(b) Divide la palabra más larga del corpus en dos mitades.

(c) Fusiona el par de símbolos adyacentes más frecuente en el corpus.

(d) Asigna a cada palabra un entero según su frecuencia.

A.2. Sin contar los tokens especiales, un tokenizador BPE a nivel de byte parte de un vocabulario de: [1 p]

(a) 128 símbolos.

(b) 256 símbolos.

(c) 65 536 símbolos.

(d) tantos símbolos como caracteres distintos aparezcan en el corpus.

A.3. ¿Cuántos bytes ocupa la palabra «piñón» codificada en UTF-8? [1 p]

(a) 5.

(b) 6.

(c) 7.

(d) 10.

A.4. Al codificar una palabra nueva con un tokenizador BPE ya entrenado, las reglas de fusión se aplican: [1 p]

(a) en el orden en que se aprendieron.

(b) de la que produce el token más largo a la que produce el más corto.

(c) en orden aleatorio, para dar variedad a la segmentación.

(d) solo la regla más frecuente.

A.5. ¿Por qué las redes recurrentes escalan peor que el Transformer? [1 p]

(a) Porque tienen más parámetros por capa.

(b) Porque no pueden usar embeddings.

(c) Porque solo admiten vocabularios de caracteres.

(d) Porque procesan los tokens uno tras otro, lo que impide paralelizar el entrenamiento a lo largo de la secuencia, y sus gradientes se desvanecen con la distancia.

A.6. En la atención de una cabeza sobre n tokens, la matriz $A = \mathrm{softmax}\big(QK^\top/\sqrt{d_k}\big)$ tiene forma: [1 p]

(a) $n \times d_k$.

(b) $d_k \times d_k$.

(c) $n \times n$.

(d) $n \times d_v$.

A.7. Si las componentes de $q, k \in \mathbb{R}^{d_k}$ son variables independientes de media 0 y varianza 1, la varianza de $q \cdot k$ es: [1 p]

(a) 1.

(b) d_k.

(c) $\sqrt{d_k}$.

(d) d_k^2.

A.8. La máscara causal se implementa: [1 p]

(a) poniendo a 0 las puntuaciones de los tokens futuros antes de la softmax.

(b) poniendo a 0 los pesos de los tokens futuros después de la softmax, sin renormalizar.

(c) eliminando de V las filas de los tokens futuros.

(d) sumando $-\infty$ a las puntuaciones de los tokens futuros antes de la softmax.

A.9. Con $d = 768$ y 12 cabezas, ¿cuántos parámetros tiene la atención multi-cabeza, sin sesgos? [1 p]

(a) 196 608.

(b) 589 824.

(c) 1 179 648.

(d) 2 359 296.

A.10. Pasar de un contexto de 1024 a uno de 8192 tokens multiplica la memoria de la matriz de atención, almacenada completa, por: [1 p]

(a) 8.

(b) 64.

(c) 512.

(d) 4096.

A.11. En un bloque pre-norm, la subcapa de atención se escribe: [1 p]

(a) $h = x + \mathrm{Attn}(\mathrm{Norm}(x))$.

(b) $h = \mathrm{Norm}(x + \mathrm{Attn}(x))$.

(c) $h = \mathrm{Norm}(\mathrm{Attn}(x))$.

(d) $h = \mathrm{Attn}(x) + \mathrm{Norm}(x)$.

A.12. ¿Por qué un Transformer pre-norm necesita una normalización final antes del unembedding? [1 p]

(a) Porque la norma del flujo residual crece con la profundidad.

(b) Porque la softmax exige entradas positivas.

(c) Para poder atar los pesos de entrada y de salida.

(d) No la necesita; solo la llevan los modelos post-norm.

A.13. Un MLP SwiGLU tiene el mismo número de parámetros que un MLP clásico de dimensión oculta $4d$ cuando su dimensión oculta vale: [1 p]

(a) $2d$.

(b) $\frac{8}{3}d$.

(c) $3d$.

(d) $4d$.

A.14. RoPE consigue que la puntuación $\tilde{q}_m \cdot \tilde{k}_n$ entre el token m y el token n: [1 p]

(a) dependa de las posiciones solo a través de $n - m$.

(b) dependa de las posiciones solo a través de $m + n$.

(c) sea independiente de las posiciones.

(d) crezca linealmente con la posición.

A.15. RoPE gira las consultas y las claves, pero no los valores, porque: [1 p]

(a) v tiene dimensión impar.

(b) girar v cambiaría su norma.

(c) la posición debe influir en a quién se atiende, no en qué contenido se copia.

(d) v no participa en la atención.

A.16. LLaMA-2 7B no ata sus embeddings ($V = 32\,000$, $d = 4096$). ¿Cuántos parámetros suman la matriz de entrada y la de salida? [1 p]

(a) 65 536 000.

(b) 131 072 000.

(c) 262 144 000.

(d) 524 288 000.

A.17. Entrenar con Adam en precisión mixta (pesos y gradientes en bf16, copia maestra en fp32 y momentos m y v en fp32) ocupa, sin contar las activaciones: [1 p]

(a) 4 bytes por parámetro.

(b) 8 bytes por parámetro.

(c) 12 bytes por parámetro.

(d) 16 bytes por parámetro.

A.18. Según la regla $6ND$, ¿cuántos FLOPs cuesta, aproximadamente, entrenar un modelo de 10^9 parámetros con $2 \cdot 10^{10}$ tokens? [1 p]

(a) $6 \cdot 10^{18}$.

(b) $2 \cdot 10^{19}$.

(c) $4 \cdot 10^{19}$.

(d) $1{,}2 \cdot 10^{20}$.

A.19. Un GPT con $V = 50\,257$ tokens, recién inicializado con logits casi nulos, debe mostrar una pérdida inicial (en nats por token) cercana a: [1 p]

(a) 10,82.

(b) 1.

(c) 0.

(d) 50 257.

A.20. Durante el entrenamiento, la pérdida de entrenamiento sigue bajando y la de validación empieza a subir. ¿Qué punto de control debe conservarse como modelo final? [1 p]

(a) El último.

(b) El de menor pérdida de validación.

(c) El de menor pérdida de entrenamiento.

(d) El primero, porque aún no ha sobreajustado.

Parte B

Problemas

La parte B consta de seis ejercicios con un total de 80 puntos. La puntuación de cada apartado figura entre paréntesis. Debe justificarse cada paso y expresarse cada resultado con sus unidades.

Módulo 14. El bloque decoder

Ejercicio B.1 ⋆⋆ **(15 puntos.)** Un Transformer decoder-only del estilo de LLaMA recibe n tokens y tiene dimensión de modelo d, L bloques, h cabezas de atención, MLP SwiGLU de dimensión oculta d_{ff}, normalización RMSNorm, codificación de posición RoPE y un vocabulario de V tokens con embeddings sin atar.

(a) (5 p) Dibuje el diagrama completo, desde los tokens enteros hasta la distribución del token siguiente, con un bloque detallado. Señale las dos conexiones residuales y las dos normalizaciones del bloque, el lugar en que entra la información de posición y la normalización final.

(b) (3 p) Escriba las ecuaciones del bloque ℓ, con entrada $h^{\ell-1}$ y salida h^{ℓ}, y la ecuación de los logits.

(c) (3 p) Indique la forma de: la entrada al primer bloque; Q tras separar las cabezas; la matriz de atención de una cabeza; la salida de la proyección W_{gate}; la salida de un bloque, y los logits.

(d) (2 p) ¿Qué subcapa mezcla información *entre* tokens y cuál procesa cada token *por separado*? ¿Por qué cada subcapa debe devolver un vector de $\mathbb{R}^d$?

(e) (2 p) Dé dos razones por las que los modelos de lenguaje actuales prefieren la variante pre-norm a la post-norm.

Fuente: adaptado de Stanford CS336 [2], tarea 1, §3.4 y §3.5, ecuación (15) y problemas *transformer_block* y *transformer_lm*, y §7.3, problema *pre_norm_ablation* (la implementación y el experimento se sustituyen por el diagrama, las ecuaciones y el análisis).

Módulo 13. Atención con máscara causal

Ejercicio B.2 ⋆⋆ **(15 puntos.)** Tres tokens tienen embeddings de dimensión $d = 2$: $x_1 = (1, 0)$, $x_2 = (0, 1)$ y $x_3 = (1, 1)$, apilados como filas de una matriz $X \in \mathbb{R}^{3\times 2}$. Una cabeza de atención usa $d_k = 4$, $d_v = 2$ y

$$W_Q = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \qquad W_K = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \end{pmatrix}, \qquad W_V = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

(a) (3 p) Calcule $Q = XW_Q$, $K = XW_K$ y $V = XW_V$.

(b) (3 p) Calcule $S = QK^\top$ y $S/\sqrt{d_k}$.

(c) (2 p) Escriba la matriz de máscara causal M y la matriz $Z = S/\sqrt{d_k} + M$. ¿Por qué la máscara usa $-\infty$ y no 0?

(d) (3 p) Calcule $A = \mathrm{softmax}(Z)$, aplicada por filas, y la salida $O = AV$.

(e) (2 p) Se sustituye x_3 por $(5, -3)$. Sin hacer cuentas largas, indique qué filas de O cambian, justificándolo con la estructura de A, y calcule la nueva fila o_3. ¿Qué hace la cabeza en este caso?

(f) (2 p) Calcule $\partial a_{22}/\partial z_{22}$ con los valores del apartado (d). ¿Qué ocurriría con esta derivada si no se dividiera por $\sqrt{d_k}$ y las puntuaciones fueran muy grandes?

Fuente: adaptado de Stanford CS224N [1], tarea 3, pregunta 1(a), «Copying in attention» (apartado e), y de Jurafsky y Martin [4], cap. 7, ejercicio 7.2 (apartado c); datos numéricos propios.

Módulo 12. BPE a mano

Ejercicio B.3 ⋆⋆ **(12 puntos.)** Un corpus contiene estas palabras con sus frecuencias: `pino` ×4, `pinos` ×2, `vino` ×3, `vinos` ×1 y `pinta` ×2. Los símbolos iniciales son los caracteres y las fusiones no cruzan los límites entre palabras.

(a) (1 p) ¿Cuántos símbolos tiene el corpus al empezar?

(b) (6 p) Aplique cuatro fusiones BPE. En cada paso indique el par elegido y su frecuencia, el par que queda en segundo lugar y el número total de símbolos del corpus tras la fusión.

(c) (3 p) Codifique con las cuatro reglas obtenidas las palabras `vinos`, `pinto`, `sino` y `ovino`, e indique el número de tokens de cada una.

(d) (2 p) Un tokenizador BPE a nivel de byte procesa la palabra «piñón». ¿Cuántos símbolos iniciales tiene? ¿Qué pares de bytes conviene fusionar pronto en un corpus en español y por qué?

Fuente: adaptado de Stanford CS336 [2], tarea 1, §2.4, ejemplo *bpe_example*, y §2.6.1, ejemplo *bpe_encoding*, y de Jurafsky y Martin [4], §2.4.1 y ejercicio 2.3 (corpus y palabras cambiados).

Módulo 14. Parámetros, FLOPs y memoria

Ejercicio B.4 ⋆⋆⋆ **(15 puntos.)** El modelo ficticio **Lince-1B** tiene $d = 2048$, $L = 24$, $h = 16$, MLP SwiGLU, $V = 48\,000$, embeddings *atados*, RMSNorm con solo vector de pesos, RoPE y ningún sesgo.

(a) (2 p) Calcule $\frac{8}{3}d$ y redondéelo al múltiplo de 256 inmediatamente superior para obtener d_{ff}.

(b) (4 p) Calcule el número de parámetros de un bloque y el total N. Compare N con la aproximación $12Ld^2$ e indique qué fracción de N corresponde a los MLP.

(c) (3 p) Estime los FLOPs por token en la generación y los FLOPs del entrenamiento con $D = 2 \cdot 10^{11}$ tokens. ¿Cuántas horas de GPU H100 ($989 \cdot 10^{12}$ FLOP/s en bf16) se necesitan con un aprovechamiento del 40 %? ¿Cuántas horas de reloj con 64 GPU?

(d) (3 p) Calcule la memoria que ocupan los pesos en bf16 y en int4, y la del estado del entrenamiento con Adam en precisión mixta. ¿Cabe el entrenamiento en una GPU de 24 GB?

(e) (3 p) La atención añade unos $4Lnd$ FLOPs por token con un contexto de n tokens. ¿Qué fracción de $2N$ supone con $n = 4096$? ¿Para qué n iguala a $2N$?

Fuente: adaptado de Stanford CS336 [2], tarea 1, §3.5, problema *transformer_accounting*, apartados (a) a (e), y §4.3, problema *adamw_accounting*, apartado (d), y de Jurafsky y Martin [4], cap. 7, ejercicio 7.1 (se sustituye GPT-2 XL por un modelo ficticio y la precisión fp32 por la mixta).

Módulos 13 y 14. Posiciones y coste de la atención

Ejercicio B.5 ⋆⋆ (11 puntos.)

(a) (5 p) Se aplica RoPE en dimensión $d_k = 2$ con $\theta = \pi/4$, $q = (1, 0)$ y $k = (1, 1)$. Use la identidad

$$q^\top R(\varphi)\, k = (q \cdot k)\cos\varphi - (q_1 k_2 - q_2 k_1)\sin\varphi, \qquad \varphi = (n - m)\theta,$$

para calcular la puntuación entre la consulta en la posición m y la clave en la posición n en los pares $(m, n) = (2, 1)$, $(5, 4)$, $(3, 3)$, $(1, 2)$ y $(0, 4)$. ¿Qué se observa en los dos primeros?

(b) (2 p) En Lince-1B (ejercicio B.4), ¿cuánto vale d_k? Si se duplica h manteniendo d, ¿cuántos parámetros de atención se ganan?

(c) (4 p) Con un contexto de $n = 32\,768$ tokens, calcule la memoria en fp16 de la matriz de atención de una cabeza y la de todas las cabezas y capas de Lince-1B. ¿A partir de qué n dominan los FLOPs cuadráticos de la atención, $4n^2 d$ por capa, sobre los de las proyecciones, $8nd^2$ por capa? ¿Qué técnica evita almacenar A completa?

Fuente: adaptado de Stanford CS336 [2], tarea 1, §3.4.3, ecuaciones (8) y (9) (RoPE), y §3.5, problema *transformer_accounting*, apartado (e) (se pide el cálculo a mano en dimensión 2 y la memoria de A).

Módulo 15. Entrenar y leer un mini-GPT

Ejercicio B.6 ⋆⋆ **(12 puntos.)** Un mini-GPT de caracteres tiene $V = 80$, contexto $T = 128$, $d = 128$, $L = 4$, $h = 4$, MLP clásico de dimensión oculta $4d$ con GELU, LayerNorm con peso y sesgo, posiciones aprendidas, embeddings atados y ninguna capa lineal con sesgo. Se entrena con lotes de $B = 32$ secuencias durante 5000 pasos sobre un corpus de 10^6 caracteres. El cuadro B.1 recoge sus pérdidas medias.

Cuadro B.1: Pérdidas del mini-GPT, en nats por carácter.

Paso	0	500	1000	1500	2000	2500	3000	3500	4000	4500	5000
Entrenamiento	4,39	2,12	1,66	1,47	1,38	1,30	1,22	1,15	1,08	1,03	0,98
Validación	4,39	2,14	1,70	1,52	1,46	1,44	1,45	1,48	1,52	1,57	1,62

(a) (3 p) Cuente los parámetros del modelo.

(b) (2 p) Explique por qué la pérdida inicial vale unos 4,39 nats.

(c) (3 p) ¿Qué punto de control debe conservarse? Calcule su perplejidad de validación y sus bits por carácter. Un modelo de bigramas alcanza una pérdida de validación de 2,45 nats: ¿cuál es su perplejidad?

(d) (2 p) ¿Cuántos tokens procesa el entrenamiento? ¿A cuántas épocas equivale? Relacione el resultado con el cuadro B.1 y proponga dos remedios.

(e) (2 p) Al generar, el modelo produce los logits $(2, 1, 0)$ para tres caracteres. Calcule las probabilidades con temperatura $\tau = 1$ y $\tau = 0{,}5$. ¿Qué efecto tiene bajar la temperatura sobre las muestras?

Fuente: adaptado de Stanford CS336 [2], tarea 1, §4.1, ecuación (18) (perplejidad), y §6, ecuación (23) y problema *decoding* (temperatura), y de Jurafsky y Martin [4], cap. 7, ejercicios 7.4 y 7.5 (curvas y logits propios).

Parte C

Proyecto de fase

Mini-GPT en español con informe técnico

Objetivo. Entrenar desde cero, en PyTorch y en CPU, un Transformer decoder-only de caracteres, y redactar un informe técnico con las curvas de pérdida, muestras generadas e interpretación de mapas de atención. El proyecto integra los cuatro módulos de la fase: el vocabulario de caracteres (M12), la atención causal multi-cabeza (M13), el bloque pre-norm (M14) y el bucle de entrenamiento y generación (M15).

Datos. Se usará un corpus en español escrito por el estudiante o generado con plantillas (frases que combinan al azar sujetos, verbos, objetos y lugares). No deben usarse textos literarios existentes, ni siquiera de dominio público. Bastan entre 300 000 y 10^6 caracteres.

Entregables

1. `H3-minigpt.ipynb`: código completo, que se ejecute en CPU en menos de 10 minutos.
2. `H3-informe.md`, de 2 a 4 páginas, con:
   - la configuración (d, L, h, T, V, B, número de pasos y tasa de aprendizaje) y el número de parámetros, contado a mano y con código;
   - las curvas de pérdida de entrenamiento y de validación en la misma gráfica, con la pérdida de un modelo de bigramas como línea de referencia y el valor $\ln V$ marcado en el paso 0;
   - tres muestras de 300 caracteres con temperaturas 0,5, 1,0 y 1,5, comentadas;
   - mapas de atención de al menos dos cabezas en dos capas sobre una frase del corpus, con su interpretación (qué cabeza atiende al carácter anterior, al inicio de palabra o a sí misma);
   - un experimento de ablación: eliminar las posiciones, usar una cabeza en lugar de cuatro o pasar de pre-norm a post-norm, comparando las pérdidas.

Pasos sugeridos

1. Generar el corpus y dividirlo en un 90 % para entrenamiento y un 10 % para validación, con un corte contiguo, no aleatorio.
2. Calcular la pérdida de validación de un modelo de bigramas con suavizado de Laplace (módulo 1): es el listón que hay que superar.

3. Implementar el bloque pre-norm con atención causal multi-cabeza y MLP. Inicializar los embeddings con desviación típica 0,02: con $\mathcal{N}(0, 1)$ y pesos atados, la pérdida inicial queda muy por encima de $\ln V$ (M15, §1.4).
4. Comprobar la causalidad: al cambiar el último carácter, los logits de las posiciones anteriores no deben cambiar (M13, §3.2).
5. Entrenar con AdamW, calentamiento lineal y decaimiento coseno, y recorte del gradiente a norma 1,0 (M15, §3.2 a §3.4). Evaluar cada 100 pasos y conservar el punto de control con menor pérdida de validación (M15, §4.2).
6. Generar muestras y extraer las matrices A de cada capa, devueltas por `forward`. Dibujarlas con `imshow`, con los caracteres en los ejes.

Orientación. En una prueba con un corpus de plantillas de 717 000 caracteres ($V = 33$), un modelo con $d = 64$, $L = 2$, $h = 4$ y $T = 32$ (103 104 parámetros) redujo la pérdida de validación de 3,50 a 0,36 nats en 800 pasos, frente a 1,75 del modelo de bigramas, y tardó menos de 30 s en CPU. Las muestras eran ya frases gramaticales del estilo de las plantillas.

Criterios de evaluación

El proyecto se califica sobre 100 con los pesos del cuadro C.1. Se considera superado con 70 o más.

Cuadro C.1: Rúbrica del proyecto de fase.

Criterio	Peso	Excelente	Suficiente	Insuficiente
Implementación del modelo	25 %	Bloque pre-norm correcto, causalidad comprobada con una prueba, recuento a mano igual al del código	Funciona y entrena; recuento con pequeñas discrepancias explicadas	Errores en la máscara o en las formas; no entrena
Entrenamiento y curvas	20 %	Curvas de entrenamiento y validación con bigramas y $\ln V$ como referencias; validación $\leq 75\,\%$ de la de bigramas; mejor punto de control justificado	Curvas correctas y validación por debajo de bigramas	Solo pérdida de entrenamiento, o no supera a bigramas
Muestras generadas	15 %	Tres temperaturas comentadas con criterio (repetición frente a incoherencia)	Muestras presentadas con un comentario breve	Sin muestras o sin comentar
Mapas de atención	20 %	Mapas legibles con ejes etiquetados, triángulo inferior visible, interpretación de cada cabeza con ejemplos	Mapas correctos con interpretación genérica	Mapas sin máscara visible o sin interpretar
Ablación	10 %	Experimento controlado (misma semilla y número de pasos) con conclusión cuantitativa	Experimento realizado con comparación cualitativa	No hay ablación
Informe	10 %	Claro y reproducible, con cifras en formato español y relación con los LLM reales	Completo pero desordenado	Faltan secciones

Apéndice A

Soluciones

Las soluciones de la parte B se presentan apartado por apartado, en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. Se indican también los criterios de corrección cuando el reparto de puntos de un apartado no es evidente. Todos los valores numéricos se han comprobado con Python; en los pasos intermedios se conservan cuatro cifras significativas.

A.1 Clave de respuestas de la parte A

Cuadro A.1: Clave de respuestas de la parte A.

N.º	Resp.	Justificación	Apuntes
A.1	(c)	BPE cuenta los pares adyacentes, ponderados por frecuencia, y fusiona el más frecuente.	M12, §2.1
A.2	(b)	Parte de los 256 valores posibles de un byte; por eso nunca aparece un token desconocido.	M12, §2.4
A.3	(c)	«p», «i» y «n» (ASCII) ocupan 1 byte cada una; «ñ» y «ó», 2 cada una: $3 \cdot 1 + 2 \cdot 2 = 7$.	M12, §1.3
A.4	(a)	Las reglas se aplican en el orden en que se aprendieron; así la codificación es determinista.	M12, §2.2
A.5	(d)	La recurrencia $h_t = f(h_{t-1}, x_t)$ exige n pasos secuenciales y el gradiente se multiplica n veces.	M12, §5.5
A.6	(c)	Cada token (fila) tiene un peso para cada token (columna).	M13, §2.1
A.7	(b)	$q \cdot k$ es la suma de d_k productos independientes de media 0 y varianza 1.	M13, §2.3
A.8	(d)	$e^{-\infty} = 0$ y los pesos permitidos siguen sumando 1; un 0 antes de la softmax no anula nada.	M13, §3.1
A.9	(d)	$4d^2 = 4 \cdot 768^2 = 2\,359\,296$, sin dependencia de h; (c) es $2d^2$.	M13, §4.2
A.10	(b)	La memoria de A crece como n^2: $(8192/1024)^2 = 8^2 = 64$.	M13, §4.4
A.11	(a)	Pre-norm normaliza la entrada de la subcapa y deja limpio el camino residual; (b) es post-norm.	M14, §2.2
A.12	(a)	En pre-norm, $h^L = h^0 + \sum$ aportaciones y su norma crece con L.	M14, §2.3
A.13	(b)	$3d\,d_{\text{ff}} = 2d \cdot 4d = 8d^2$, luego $d_{\text{ff}} = \frac{8}{3}d$.	M14, §3.3
A.14	(a)	$R_m^\top R_n = R_{n-m}$, luego $\tilde{q}_m \cdot \tilde{k}_n = q^\top R_{n-m} k$.	M14, §4.3
A.15	(c)	v es el contenido que se copia; la posición solo debe decidir los pesos.	M14, §4.3
A.16	(c)	$2Vd = 2 \cdot 32\,000 \cdot 4096 = 262\,144\,000$.	M14, §1.4 y §5.2
A.17	(d)	$2 + 2 + 4 + 4 + 4 = 16$ bytes por parámetro.	M14, §5.3
A.18	(d)	$6ND = 6 \cdot 10^9 \cdot 2 \cdot 10^{10} = 1{,}2 \cdot 10^{20}$.	M14, §6.3
A.19	(a)	Logits iguales dan la distribución uniforme y la pérdida $\ln V = \ln 50\,257 = 10{,}82$.	M15, §1.4
A.20	(b)	La pérdida de validación estima el rendimiento sobre datos nuevos.	M15, §4.2 y §4.4

A.2 Soluciones de la parte B

Solución del ejercicio B.1

Apartado (a) (5 p).

Datos. Transformer decoder-only del estilo de LLaMA: n tokens, dimensión d, L bloques, h cabezas, SwiGLU con d_{ff}, RMSNorm, RoPE y V tokens sin atar.

Planteamiento. recorrer el camino de los datos: enteros, embedding, L bloques pre-norm, normalización final, unembedding y softmax (M14, §1.2).

Desarrollo.

1. Entrada: los n enteros seleccionan filas de $W_E \in \mathbb{R}^{V\times d}$ y forman $h^0 \in \mathbb{R}^{n\times d}$.
2. Primera subcapa del bloque: RMSNorm 1, proyecciones Q, K, V, rotación RoPE de q y k (aquí entra la posición), atención causal en cada cabeza, concatenación y W_O, y suma residual 1.
3. Segunda subcapa: RMSNorm 2, SwiGLU con W_{gate}, W_{up} y W_{down}, y suma residual 2.
4. Tras los L bloques: RMSNorm final, $W_U \in \mathbb{R}^{d\times V}$ y softmax.

Resultado. **el diagrama de la figura A.1**, con dos residuales por bloque, una normalización antes de cada subcapa, RoPE aplicada a q y k y una RMSNorm final.

Comprobación. todas las flechas entre bloques transportan matrices $n \times d$; solo cambian de forma la entrada (enteros a $\mathbb{R}^d$) y la salida ($\mathbb{R}^d$ a $\mathbb{R}^V$).

Criterios: embedding y unembedding con sus formas (1 p); dos residuales que puentean cada subcapa (1 p); normalizaciones antes de cada subcapa y normalización final (1 p); atención causal con máscara y RoPE en q y k (1 p); MLP con sus tres matrices (1 p). Si se dibujan posiciones aprendidas sumadas al embedding, al estilo de GPT-2, se acepta siempre que se indique.

Apartado (b) (3 p).

Datos. entrada del bloque $h^{\ell-1}$; arquitectura pre-norm.

Planteamiento. en pre-norm, cada subcapa lee su entrada normalizada y suma su salida al flujo residual (M14, §2.2).

Desarrollo.

1. Atención: $a^\ell = h^{\ell-1} + \mathrm{MHA}\big(\mathrm{RMSNorm}_1(h^{\ell-1})\big)$.
2. MLP: $h^\ell = a^\ell + \mathrm{SwiGLU}\big(\mathrm{RMSNorm}_2(a^\ell)\big)$.
3. Salida: $\text{logits} = \mathrm{RMSNorm}_f(h^L)\, W_U$.

Resultado. **las tres ecuaciones anteriores**, 1 p cada una.

Comprobación. desarrollando la recurrencia, $h^L = h^0 + \sum_\ell \big(\text{aportación de la atención} + \text{aportación del MLP}\big)$: el camino identidad no atraviesa ninguna normalización (M14, §2.1).

Apartado (c) (3 p).

Datos. n tokens, d, h cabezas, d_{ff}, V.

Planteamiento. seguir las formas en cada producto $Y = XW$; 0,5 p por forma.

Desarrollo.

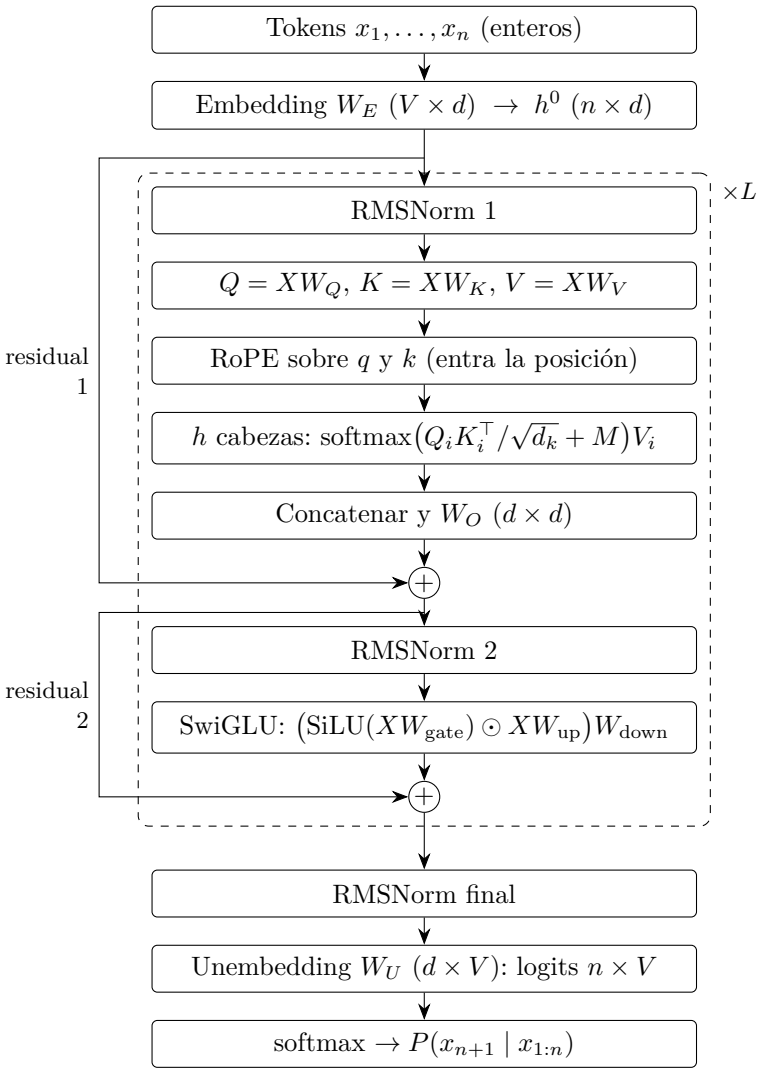


Figura A.1: Transformer decoder-only con bloques pre-norm del estilo de LLaMA. El recuadro discontinuo es un bloque y se repite L veces.

1. Entrada al primer bloque: n filas de W_E, es decir, $n \times d$.
2. $Q = XW_Q$ es $n \times d$; separada en h cabezas de dimensión d/h: $h \times n \times (d/h)$.
3. Atención de una cabeza: $(n \times d/h)(d/h \times n)$, es decir, $n \times n$.
4. $W_{\mathrm{gate}} \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$: $(n \times d)(d \times d_{\mathrm{ff}})$ da $n \times d_{\mathrm{ff}}$.
5. Salida de un bloque: $n \times d$, la misma forma que la entrada.
6. Logits: $(n \times d)(d \times V)$ da $n \times V$.

Resultado. $n \times d$**;** $h \times n \times (d/h)$**;** $n \times n$**;** $n \times d_{\mathrm{ff}}$**;** $n \times d$**;** $n \times V$**.**

Comprobación. como $h \cdot (d/h) = d$, la concatenación de las cabezas recupera la forma $n \times d$.

Apartado (d) (2 p).

Datos. subcapas de atención y MLP.

Planteamiento. comprobar si la operación combina filas distintas de la matriz $n \times d$ (M14, §3.1).

Desarrollo.

1. La atención mezcla tokens: la fila i de $O = AV$ es $\sum_{j \le i} a_{ij} v_j$.
2. El MLP aplica la misma función a cada fila por separado: la salida del token i solo depende de la fila i.
3. Cada subcapa *suma* su salida al flujo residual, que vive en $\mathbb{R}^d$; la suma exige la misma dimensión, y por eso $W_O \in \mathbb{R}^{d \times d}$ y $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$ devuelven a $\mathbb{R}^d$.

Resultado. **la atención mezcla información entre tokens; el MLP actúa token a token; ambas devuelven $\mathbb{R}^d$ porque se suman al flujo residual.**

Comprobación. si W_{down} fuera $d_{\text{ff}} \times d_{\text{ff}}$, la suma $a^\ell + \text{SwiGLU}(\cdot)$ no estaría definida.

Apartado (e) (2 p).

Datos. pre-norm, $h = x + F(\text{Norm}(x))$, frente a post-norm, $h = \text{Norm}(x + F(x))$.

Planteamiento. comparar el camino del gradiente y la estabilidad del entrenamiento (M14, §2.2).

Desarrollo.

1. En pre-norm existe un camino identidad de la entrada a la salida: el jacobiano $\partial h^L / \partial h^0$ contiene el término I y el gradiente llega a las primeras capas sin atravesar normalizaciones.
2. El entrenamiento es estable con muchas capas y con un calentamiento corto o nulo; en post-norm, el gradiente de las últimas capas es grande en la inicialización y obliga a un calentamiento largo y cuidadoso [12].

Resultado. **gradiente limpio por el camino identidad y entrenamiento estable en profundidad, poco sensible al calentamiento**, 1 p por razón.

Comprobación. GPT-2 [6], LLaMA-2 [10] y la tarea 1 de CS336 [2, §3.4] usan pre-norm; el problema *pre_norm_ablation* de esa tarea compara ambas variantes experimentalmente.

Repasar: M14, §1.2, §2.1, §2.2, §2.3 y §3.1.

Solución del ejercicio B.2

Apartado (a) (3 p).

Datos. X con filas $x_1 = (1, 0)$, $x_2 = (0, 1)$, $x_3 = (1, 1)$; $W_Q, W_K \in \mathbb{R}^{2 \times 4}$, $W_V \in \mathbb{R}^{2 \times 2}$.

Planteamiento. la fila i de XW es $x_i W$: x_1 selecciona la primera fila de W, x_2 la segunda y x_3 suma ambas (M13, §2.1).

Desarrollo.

1. Q: $(1, 1, 0, 0)$, $(0, 0, 1, 1)$ y su suma $(1, 1, 1, 1)$.
2. K: $(2, 0, 0, 0)$, $(0, 0, 2, 0)$ y su suma $(2, 0, 2, 0)$.
3. V: $(1, 1)$, $(0, 1)$ y su suma $(1, 2)$.

Resultado.

$$Q = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 2 & 0 & 2 & 0 \end{pmatrix}, \qquad V = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix},$$

1 p cada matriz.

Comprobación. en las tres matrices, la fila 3 es la suma de las filas 1 y 2, porque $x_3 = x_1 + x_2$.

Apartado (b) (3 p).

Datos. Q y K del apartado (a); $d_k = 4$.

Planteamiento. $S_{ij} = q_i \cdot k_j$; después se divide por $\sqrt{d_k} = 2$.

Desarrollo.

1. Fila 1, $q_1 = (1, 1, 0, 0)$: $q_1 \cdot k_1 = 2$, $q_1 \cdot k_2 = 0$, $q_1 \cdot k_3 = 2$.
2. Fila 2, $q_2 = (0, 0, 1, 1)$: $q_2 \cdot k_1 = 0$, $q_2 \cdot k_2 = 2$, $q_2 \cdot k_3 = 2$.
3. Fila 3, $q_3 = (1, 1, 1, 1)$: $q_3 \cdot k_1 = 2$, $q_3 \cdot k_2 = 2$, $q_3 \cdot k_3 = 4$.

Resultado.

$$S = \begin{pmatrix} 2 & 0 & 2 \\ 0 & 2 & 2 \\ 2 & 2 & 4 \end{pmatrix}, \qquad \frac{S}{\sqrt{d_k}} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix};$$

2 p por S y 1 p por el escalado.

Comprobación. la columna 3 de S es la suma de las columnas 1 y 2, porque $k_3 = k_1 + k_2$.

Apartado (c) (2 p).

Datos. tres posiciones; máscara causal.

Planteamiento. $M_{ij} = 0$ si $j \le i$ y $M_{ij} = -\infty$ si $j > i$ (M13, §3.1).

Desarrollo.

1. M es triangular superior estricta de $-\infty$; se suma elemento a elemento a $S/\sqrt{d_k}$:

$$M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & -\infty & -\infty \\ 0 & 1 & -\infty \\ 1 & 1 & 2 \end{pmatrix}.$$

2. Con $-\infty$, $e^{-\infty} = 0$: el peso de cada token futuro es exactamente nulo y los pesos permitidos se renormalizan solos. Con 0, $e^0 = 1$ y el futuro conservaría peso; por ejemplo, la fila 1 sería softmax$(1, 0, 1) = (0{,}422;\ 0{,}155;\ 0{,}422)$.

Resultado. M **y** Z **como arriba;** $-\infty$ **anula exactamente los pesos futuros**, mientras que 0 no anula nada.

Comprobación. la fila i de Z conserva i valores finitos: 1, 2 y 3.

Apartado (d) (3 p).

Datos. Z del apartado (c); $e \approx 2{,}718$, $e^2 \approx 7{,}389$; V del apartado (a).

Planteamiento. softmax por filas con $e^{-\infty} = 0$; después $O = AV$.

Desarrollo.

1. Fila 1: solo z_{11} es finito, luego $a_1 = (1, 0, 0)$.
2. Fila 2: $e^0 = 1$ y $e^1 = 2{,}718$, con suma 3,718: $a_{21} = 0{,}269$ y $a_{22} = 0{,}731$.
3. Fila 3: dividiendo e^1, e^1, e^2 por e quedan $1, 1, e$, con suma 4,718: $a_{31} = a_{32} = 0{,}212$ y $a_{33} = 0{,}576$.

4. $o_1 = (1, 1)$; $o_2 = 0{,}269\,(1, 1) + 0{,}731\,(0, 1) = (0{,}269;\ 1)$.
5. $o_3 = 0{,}212\,(1, 1) + 0{,}212\,(0, 1) + 0{,}576\,(1, 2) = (0{,}788;\ 1{,}576)$, con los pesos a cuatro cifras (0,2119 y 0,5761).

Resultado.

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0{,}269 & 0{,}731 & 0 \\ 0{,}212 & 0{,}212 & 0{,}576 \end{pmatrix}, \qquad O = AV = \begin{pmatrix} 1 & 1 \\ 0{,}269 & 1 \\ 0{,}788 & 1{,}576 \end{pmatrix}.$$

Comprobación. cada fila de A suma 1; como la segunda componente de v_1 y de v_2 es 1, la de o_2 vale exactamente 1. Criterios: máscara aplicada antes de la softmax (1 p), A correcta (1 p), O correcta (1 p).

Apartado (e) (2 p).

Datos. $x_3 = (5, -3)$; el resto, igual.

Planteamiento. A es triangular inferior: o_1 y o_2 solo dependen de los tokens 1 y 2 (M13, §3.2). Solo hay que recalcular la fila 3.

Desarrollo.

1. o_1 y o_2 no dependen de x_3: solo cambia o_3.
2. $q_3 = 5\,(1, 1, 0, 0) - 3\,(0, 0, 1, 1) = (5, 5, -3, -3)$; $k_3 = 5\,(2, 0, 0, 0) - 3\,(0, 0, 2, 0) = (10, 0, -6, 0)$; $v_3 = 5\,(1, 1) - 3\,(0, 1) = (5, 2)$.
3. Puntuaciones escaladas: $q_3 \cdot k_1/2 = 10/2 = 5$; $q_3 \cdot k_2/2 = -6/2 = -3$; $q_3 \cdot k_3/2 = (50 + 18)/2 = 34$.
4. $e^{34}/e^5 = e^{29} \approx 3{,}9 \cdot 10^{12}$, luego $a_{33} \approx 1$, $a_{31} \approx e^{-29} \approx 2{,}5 \cdot 10^{-13}$ y $a_{32} \approx e^{-37} \approx 8{,}5 \cdot 10^{-17}$.
5. $o_3 \approx v_3 = (5, 2)$.

Resultado. **solo cambia o_3, que pasa a ser** (5,000; 2,000): la cabeza *copia* el valor v_3, porque la puntuación de k_3 es mucho mayor que las demás.

Comprobación. con Python, $1 - a_{33} = 2{,}5 \cdot 10^{-13}$. Es la situación de «copia en la atención» de la tarea 3 de CS224N [1]: casi todo el peso recae en un solo elemento cuando $q \cdot k_j \gg q \cdot k_i$ para todo $i \neq j$.

Apartado (f) (2 p).

Datos. $a_{22} = 0{,}731$, del apartado (d).

Planteamiento. para la softmax, $\partial a_{ij}/\partial z_{ij} = a_{ij}(1 - a_{ij})$; el escalado por $\sqrt{d_k}$ evita la saturación [11, §3.2.1] (M13, §2.3).

Desarrollo.

1. $\partial a_{22}/\partial z_{22} = 0{,}731 \cdot 0{,}269 = 0{,}197$.
2. Si las puntuaciones crecen sin escalar, la softmax se satura: $a_{ij} \to 0$ o $a_{ij} \to 1$, y en ambos casos $a_{ij}(1 - a_{ij}) \to 0$.
3. Sin gradiente, la cabeza deja de aprender a quién atender.

Resultado. $\partial a_{22}/\partial z_{22} = 0{,}197$**; con puntuaciones grandes tiende a 0 (softmax saturada).**

Comprobación. el máximo de $a(1-a)$ es 0,25, en $a = 0{,}5$, y $0{,}197 < 0{,}25$. Respecto de la puntuación sin escalar $s = 2z$, la regla de la cadena añade el factor $1/\sqrt{d_k} = 1/2$: 0,098.

Repasar: M13, §2.1, §2.3, §3.1 y §3.2.

Solución del ejercicio B.3

Apartado (a) (1 p).

Datos. `pino` ×4, `pinos` ×2, `vino` ×3, `vinos` ×1, `pinta` ×2.

Planteamiento. número de símbolos = $\sum$(longitud × frecuencia).

Desarrollo.

1. $4 \cdot 4 + 5 \cdot 2 + 4 \cdot 3 + 5 \cdot 1 + 5 \cdot 2 = 16 + 10 + 12 + 5 + 10$.

Resultado. **53 símbolos.**

Comprobación. hay $4 + 2 + 3 + 1 + 2 = 12$ palabras de 4 o 5 letras: el total debe estar entre 48 y 60.

Apartado (b) (6 p).

Datos. el corpus del apartado (a), partido en caracteres.

Planteamiento. en cada paso se cuentan los pares adyacentes ponderados por la frecuencia de la palabra, se fusiona el más frecuente y se restan sus apariciones del total de símbolos [7] (M12, §2.1). 1,5 p por paso.

Desarrollo.

1. Recuento inicial: (`i`, `n`) = 4 + 2 + 3 + 1 + 2 = 12; (`n`, `o`) = 4 + 2 + 3 + 1 = 10; (`p`, `i`) = 4 + 2 + 2 = 8; (`v`, `i`) = 4; (`o`, `s`) = 3. Gana (`i`, `n`); símbolos: 53 − 12 = 41.
2. Tras `in`: (`in`, `o`) = 10; (`p`, `in`) = 8; (`v`, `in`) = 4; (`o`, `s`) = 3. Gana (`in`, `o`); símbolos: 41 − 10 = 31.
3. Tras `ino`: (`p`, `ino`) = 4 + 2 = 6; (`v`, `ino`) = 4; (`ino`, `s`) = 3. Gana (`p`, `ino`); símbolos: 31 − 6 = 25.
4. Tras `pino`: (`v`, `ino`) = 4; empatan con 2 (`pino`, `s`), (`p`, `in`), (`in`, `t`) y (`t`, `a`). Gana (`v`, `ino`); símbolos: 25 − 4 = 21.

Paso	Par (frec.)	Segundo (frec.)	Corpus tras la fusión	Símbolos
1	`i n` (12)	`n o` (10)	`p in o`×4, `p in o s`×2, `v in o`×3, `v in o s`×1, `p in t a`×2	41
2	`in o` (10)	`p in` (8)	`p ino`×4, `p ino s`×2, `v ino`×3, `v ino s`×1, `p in t a`×2	31
3	`p ino` (6)	`v ino` (4)	`pino`×4, `pino s`×2, `v ino`×3, `v ino s`×1, `p in t a`×2	25
4	`v ino` (4)	cuatro pares (2)	`pino`×4, `pino s`×2, `vino`×3, `vino s`×1, `p in t a`×2	21

Resultado. **reglas, en orden:** `in`**,** `ino`**,** `pino`**,** `vino`**; el corpus queda con 21 símbolos.**

Comprobación. recuento directo: $1 \cdot 4 + 2 \cdot 2 + 1 \cdot 3 + 2 \cdot 1 + 4 \cdot 2 = 21$. No hay empates en el primer puesto, de modo que la regla de desempate (M12, §2.3) no interviene. Error frecuente: tomar $(\texttt{p}, \texttt{i}) = 8$ como máximo olvidando las apariciones en `vino`.

Apartado (c) (3 p).

Datos. reglas en orden: $(\texttt{i}, \texttt{n})$, $(\texttt{in}, \texttt{o})$, $(\texttt{p}, \texttt{ino})$, $(\texttt{v}, \texttt{ino})$.

Planteamiento. partir cada palabra en caracteres y aplicar las reglas en el orden aprendido (M12, §2.2).

Desarrollo.

1. `vinos`: `v i n o s` → `v in o s` → `v ino s` → `vino s`: 2 tokens.
2. `pinto`: `p i n t o` → `p in t o`; la regla 2 no se aplica (tras `in` viene `t`) y la 3 necesita `ino`: 4 tokens.
3. `sino`: `s i n o` → `s in o` → `s ino`; no existe la regla $(\texttt{s}, \texttt{ino})$: 2 tokens.
4. `ovino`: `o v i n o` → `o v in o` → `o v ino` → `o vino`: 2 tokens.

Resultado. `vinos` = [`vino`, `s`] **(2);** `pinto` = [`p`, `in`, `t`, `o`] **(4);** `sino` = [`s`, `ino`] **(2);** `ovino` = [`o`, `vino`] **(2).**

Comprobación. ninguna palabra nueva produce un token desconocido, porque los caracteres sueltos pertenecen siempre al vocabulario.

Apartado (d) (2 p).

Datos. «piñón» en UTF-8.

Planteamiento. los caracteres ASCII ocupan 1 byte; «ñ» y «ó», 2 (M12, §1.3 y §1.5).

Desarrollo.

1. Bytes en hexadecimal: `70`, `69`, `C3 B1`, `C3 B3`, `6E`: $1 + 1 + 2 + 2 + 1 = 7$.
2. En español, los pares $(\texttt{C3}, \texttt{B1})$, que forman «ñ», y $(\texttt{C3}, \texttt{B3})$, que forma «ó» (y análogos como $(\texttt{C3}, \texttt{A1})$ para «á»), son muy frecuentes.
3. Sin esas fusiones, ninguna palabra con tilde o con «ñ» puede llegar a ser un único token.

Resultado. **7 símbolos iniciales; conviene fusionar pronto** $(\texttt{C3}, \texttt{B1})$ **y** $(\texttt{C3}, \texttt{B3})$, los dos bytes de «ñ» y de «ó».

Comprobación. coincide con la cuestión A.3 (7 bytes).

Repasar: M12, §1.3, §1.5, §2.1, §2.2 y §2.4.

Solución del ejercicio B.4

Apartado (a) (2 p).

Datos. $d = 2048$.

Planteamiento. $d_{\text{ff}} = \frac{8}{3}d$ redondeado hacia arriba al múltiplo de 256 [8] (M14, §3.3).

Desarrollo.

1. $8 \cdot 2048/3 = 16\,384/3 = 5461{,}33$.
2. $5461{,}33/256 = 21{,}33$; el entero siguiente es 22.

3. $22 \cdot 256 = 5632$.

Resultado. $d_{\text{ff}} = 5632$**.**

Comprobación. $21 \cdot 256 = 5376 < 5461{,}33 < 5632$.

Apartado (b) (4 p).

Datos. $d = 2048$, $L = 24$, $d_{\text{ff}} = 5632$, $V = 48\,000$; embeddings atados; RMSNorm solo con pesos; sin sesgos.

Planteamiento. $N = L\,(4d^2 + 3d\,d_{\text{ff}} + 2d) + Vd + d$, con una sola matriz de embeddings por estar atados (M14, §5.1 y §5.2).

Desarrollo.

1. $d^2 = 4\,194\,304$; atención: $4d^2 = 16\,777\,216$.
2. MLP: $3 \cdot 2048 \cdot 5632 = 34\,603\,008$.
3. Dos RMSNorm: $2 \cdot 2048 = 4096$. Bloque: $16\,777\,216 + 34\,603\,008 + 4096 = 51\,384\,320$.
4. 24 bloques: $24 \cdot 51\,384\,320 = 1\,233\,223\,680$.
5. Embedding: $48\,000 \cdot 2048 = 98\,304\,000$; RMSNorm final: 2048.
6. $N = 1\,233\,223\,680 + 98\,304\,000 + 2048 = 1\,331\,529\,728$.
7. $12Ld^2 = 12 \cdot 24 \cdot 4\,194\,304 = 1\,207\,959\,552$, que es el 90,7 % de N.
8. MLP: $24 \cdot 34\,603\,008 = 830\,472\,192$, el 62,4 % de N.

Pieza	Fórmula	Parámetros
Atención	$4d^2$	16 777 216
MLP SwiGLU	$3d\,d_{\text{ff}}$	34 603 008
Dos RMSNorm	$2d$	4096
Bloque		**51 384 320**

Resultado. **bloque:** 51 384 320**;** $N = 1\,331\,529\,728 \approx 1{,}33 \cdot 10^9$**;** $12Ld^2$ **es el** 90,7 % **de** N**; los MLP suponen el** 62,4 %**.**

Comprobación. la diferencia con la aproximación procede del embedding ($98\,304\,000/N = 7{,}4\,\%$) y del redondeo de d_{ff} hacia arriba. Sin atar, N sería 1 429 833 728.

Apartado (c) (3 p).

Datos. $N = 1{,}3315 \cdot 10^9$; $D = 2 \cdot 10^{11}$; H100 a $989 \cdot 10^{12}$ FLOP/s; aprovechamiento del 40 %; 64 GPU.

Planteamiento. inferencia, $\approx 2N$ FLOPs por token; entrenamiento, $C \approx 6ND$ [5]; tiempo $= C/$velocidad efectiva (M14, §6.2 y §6.3).

Desarrollo.

1. Inferencia: $2 \cdot 1{,}3315 \cdot 10^9 = 2{,}663 \cdot 10^9$ FLOPs por token.
2. Entrenamiento: $C = 6 \cdot 1{,}3315 \cdot 10^9 \cdot 2 \cdot 10^{11} = 1{,}598 \cdot 10^{21}$ FLOPs.
3. Velocidad efectiva por GPU: $0{,}4 \cdot 989 \cdot 10^{12} = 3{,}956 \cdot 10^{14}$ FLOP/s.
4. Tiempo: $1{,}598 \cdot 10^{21}/3{,}956 \cdot 10^{14} = 4{,}039 \cdot 10^6$ s, es decir, 1122 h de GPU.

5. Con 64 GPU: $1122/64 = 17{,}53$ h de reloj (0,73 días).

Resultado. $2{,}66 \cdot 10^9$ **FLOPs por token;** $C \approx 1{,}60 \cdot 10^{21}$ **FLOPs; unas** 1122 **h de H100;** 17,5 **h de reloj con 64 GPU.**

Comprobación. LLaMA-2 7B [10] ($N = 6{,}74 \cdot 10^9$, $D = 2 \cdot 10^{12}$) cuesta $8{,}09 \cdot 10^{22}$ FLOPs, unas 50,6 veces más, coherente con un N cinco veces mayor y un D diez veces mayor.

Apartado (d) (3 p).

Datos. $N = 1\,331\,529\,728$; bf16, 2 bytes; int4, 0,5 bytes; Adam en precisión mixta, 16 bytes por parámetro.

Planteamiento. memoria = $N\times$ bytes por parámetro (M14, §5.3).

Desarrollo.

1. bf16: $2N = 2{,}663 \cdot 10^9$ bytes = 2,66 GB.
2. int4: $0{,}5N = 6{,}658 \cdot 10^8$ bytes = 0,67 GB.
3. Adam en precisión mixta: $16N = 2{,}130 \cdot 10^{10}$ bytes = 21,3 GB.
4. $21{,}3 < 24$: los estados caben, pero solo quedan unos 2,7 GB para las activaciones.

Resultado. 2,66 **GB en bf16,** 0,67 **GB en int4 y** 21,3 **GB para el estado del entrenamiento.** El estado cabe en 24 GB, pero las activaciones, que crecen con el lote y el contexto, no caben con holgura.

Comprobación. $21{,}3/2{,}66 = 8 = 16/2$. En la práctica se recurriría a la recomputación de activaciones o al reparto entre varias GPU (ZeRO, módulo 17).

Apartado (e) (3 p).

Datos. $L = 24$, $d = 2048$, $n = 4096$; $2N = 2{,}663 \cdot 10^9$.

Planteamiento. comparar $4Lnd$ con $2N$ y despejar n en la igualdad: $n^* = N/(2Ld)$ (M14, §6.2).

Desarrollo.

1. $4 \cdot 24 \cdot 4096 \cdot 2048 = 805\,306\,368 \approx 8{,}053 \cdot 10^8$ FLOPs por token.
2. Fracción: $8{,}053 \cdot 10^8 / 2{,}663 \cdot 10^9 = 0{,}302$.
3. Igualdad: $n^* = 2N/(4Ld) = 2{,}663 \cdot 10^9 / 196\,608 = 13\,545$.

Resultado. **la atención añade un** 30,2 % **a** $2N$ **con** $n = 4096$**, y lo iguala con** $n \approx 13\,545$ **tokens.**

Comprobación. la fracción es lineal en n: $4096/13\,545 = 0{,}302$.

Repasar: M14, §3.3, §5.1, §5.2, §5.3, §6.2 y §6.3.

Solución del ejercicio B.5

Apartado (a) (5 p).

Datos. $d_k = 2$, $\theta = \pi/4$, $q = (1, 0)$, $k = (1, 1)$; $\cos(\pi/4) = \sin(\pi/4) = 0{,}707$.

Planteamiento. calcular las dos constantes de la identidad y evaluar en cada $\varphi = (n - m)\theta$ [9] (M14, §4.3).

Desarrollo.

1. $q \cdot k = 1$ y $q_1 k_2 - q_2 k_1 = 1$; la puntuación es $\cos\varphi - \sin\varphi$.
2. $(2,1)$ y $(5,4)$: $n - m = -1$, $\varphi = -\pi/4$, puntuación $0{,}707 + 0{,}707 = 1{,}414$.
3. $(3,3)$: $\varphi = 0$, puntuación 1.
4. $(1,2)$: $\varphi = \pi/4$, puntuación 0.
5. $(0,4)$: $\varphi = \pi$, puntuación -1.

(m,n)	$n-m$	φ	$\cos\varphi - \sin\varphi$
$(2,1)$	-1	$-\pi/4$	1,414
$(5,4)$	-1	$-\pi/4$	1,414
$(3,3)$	0	0	1
$(1,2)$	$+1$	$\pi/4$	0
$(0,4)$	$+4$	π	-1

Resultado. 1,414**;** 1,414**;** 1**;** 0**;** -1**.** Los dos primeros pares ocupan posiciones distintas a la misma distancia y dan la misma puntuación: RoPE codifica la posición relativa.

Comprobación. con distancia nula se recupera $q \cdot k = 1$; además, $\cos\varphi - \sin\varphi = \sqrt{2}\cos(\varphi + \pi/4)$, cuyo máximo, $\sqrt{2} = 1{,}414$, se alcanza en $\varphi = -\pi/4$. Criterios: 1 p por las dos primeras filas, 1 p por cada una de las otras tres; la observación es necesaria para obtener la puntuación completa.

Apartado (b) (2 p).

Datos. $d = 2048$, $h = 16$.

Planteamiento. $d_k = d/h$; los parámetros de la atención son $4d^2$, sin dependencia de h (M13, §4.2).

Desarrollo.

1. $d_k = 2048/16 = 128$.
2. Con $h = 32$, $d_k = 64$, pero las 32 cabezas siguen formando W_Q, W_K, W_V y W_O de tamaño $d \times d$: $4d^2 = 16\,777\,216$ en ambos casos.

Resultado. $d_k = 128$**; duplicar** h **no añade parámetros.**

Comprobación. coincide con la cuestión A.9: el recuento $4d^2$ no depende de h.

Apartado (c) (4 p).

Datos. $n = 32\,768$; fp16, 2 bytes; $h = 16$; $L = 24$; $d = 2048$.

Planteamiento. cada matriz A es $n \times n$; FLOPs cuadráticos $4n^2 d$ frente a lineales $8nd^2$ (M13, §4.3 y §4.4).

Desarrollo.

1. Una cabeza: $n^2 = 1\,073\,741\,824$ elementos; por 2 bytes, $2{,}147 \cdot 10^9$ bytes $= 2{,}15$ GB.
2. Todas las cabezas y capas: $2{,}147 \cdot 10^9 \cdot 16 \cdot 24 = 8{,}246 \cdot 10^{11}$ bytes ≈ 825 GB.
3. Cruce: $4n^2 d > 8nd^2$ equivale, dividiendo por $4nd > 0$, a $n > 2d = 4096$.
4. FlashAttention calcula la softmax por bloques y nunca almacena A completa: la memoria adicional es lineal en n, aunque el número de FLOPs sigue siendo $O(n^2 d)$ [3].

Resultado. 2,15 **GB por cabeza y unos 825 GB en total; la atención domina en FLOPs a partir de $n = 4096$; FlashAttention evita almacenar A.**

Comprobación. 825 GB supera con mucho los 80 GB de una GPU actual: sin FlashAttention, ese contexto es inviable.

Repasar: M13, §4.2, §4.3 y §4.4; M14, §4.3.

Solución del ejercicio B.6

Apartado (a) (3 p).

Datos. $V = 80$, $T = 128$, $d = 128$, $L = 4$; MLP de $4d$; LayerNorm con peso y sesgo; posiciones aprendidas; embeddings atados; lineales sin sesgo.

Planteamiento. bloque: $4d^2+8d^2+2\cdot 2d$; fuera de los bloques: Vd (una vez), Td y la LayerNorm final, $2d$ (M15, §2.3).

Desarrollo.

1. $d^2 = 16\,384$; atención: 65 536; MLP: $8d^2 = 131\,072$.
2. Dos LayerNorm: $2 \cdot 2 \cdot 128 = 512$. Bloque: 197 120.
3. Cuatro bloques: 788 480.
4. Embedding: $80 \cdot 128 = 10\,240$; posiciones: $128 \cdot 128 = 16\,384$; LayerNorm final: 256.
5. $N = 788\,480 + 10\,240 + 16\,384 + 256 = 815\,360$.

Resultado. $N = 815\,360$ **parámetros.**

Comprobación. $12Ld^2 = 786\,432$, el 96,5 % de N.

Apartado (b) (2 p).

Datos. $V = 80$; pérdida inicial $\approx 4{,}39$.

Planteamiento. logits casi iguales dan una softmax casi uniforme y una pérdida $\ln V$ (M15, §1.4).

Desarrollo.

1. Con pesos pequeños, los logits son casi iguales y $p_i \approx 1/80$.
2. $-\ln(1/80) = \ln 80 = \ln 8 + \ln 10 = 2{,}079 + 2{,}303 = 4{,}382$.

Resultado. $\ln 80 = 4{,}382 \approx 4{,}39$**.**

Comprobación. una pérdida inicial mucho mayor indicaría una inicialización defectuosa, por ejemplo embeddings $\mathcal{N}(0, 1)$ atados.

Apartado (c) (3 p).

Datos. cuadro B.1; bigramas, 2,45; $\ln 2 = 0{,}693$.

Planteamiento. se elige el mínimo de validación; PPL $= e^{\ell}$ y bits por carácter $= \ell / \ln 2$ (M15, §1.3 y §4.2).

Desarrollo.

1. Mínimo de validación: paso 2500, con 1,44 (antes, 1,46; después, 1,45).
2. PPL $= e^{1{,}44} = 4{,}221$.

3. Bits por carácter: $1{,}44/0{,}693 = 2{,}078$.
4. Bigramas: $e^{2{,}45} = 11{,}59$; cociente $11{,}59/4{,}221 = 2{,}75$.

Resultado. **punto de control del paso 2500;** PPL $\approx 4{,}22$ **y** $2{,}08$ **bits por carácter; el modelo de bigramas tiene** PPL $\approx 11{,}59$: el mini-GPT duda entre casi tres veces menos opciones.

Comprobación. $2^{2{,}078} = 4{,}22$: la perplejidad no depende de la base del logaritmo.

Apartado (d) (2 p).

Datos. $B = 32$, $T = 128$, 5000 pasos; corpus de 10^6 caracteres.

Planteamiento. tokens $= B \cdot T \cdot$ pasos; épocas = tokens / tamaño del corpus (M15, §1.2 y §4.4).

Desarrollo.

1. $32 \cdot 128 = 4096$ tokens por paso; por 5000 pasos, 20 480 000 tokens.
2. Épocas: $20\,480\,000/10^6 = 20{,}48$.
3. Brecha entre validación y entrenamiento: $1{,}44 - 1{,}30 = 0{,}14$ en el paso 2500 y $1{,}62 - 0{,}98 = 0{,}64$ en el 5000.
4. El modelo ve cada carácter unas 20 veces y memoriza: la validación sube desde el paso 2500.

Resultado. 20 480 000 **tokens, unas** 20,5 **épocas: sobreajuste.** Remedios: más datos (el mejor), parada temprana, *dropout*, más decaimiento de pesos o un modelo más pequeño.

Comprobación. la pérdida de entrenamiento baja mientras la de validación sube: es la firma del sobreajuste en la tabla de síntomas de M15, §4.4.

Apartado (e) (2 p).

Datos. logits $(2, 1, 0)$; $\tau = 1$ y $\tau = 0{,}5$.

Planteamiento. $p = \mathrm{softmax}(z/\tau)$ (M15, §5.2).

Desarrollo.

1. $\tau = 1$: $e^2 = 7{,}389$, $e^1 = 2{,}718$, $e^0 = 1$, con suma 11,107: $p = (0{,}665;\ 0{,}245;\ 0{,}090)$.
2. $\tau = 0{,}5$: $z/\tau = (4, 2, 0)$; $e^4 = 54{,}60$, $e^2 = 7{,}389$, $e^0 = 1$, con suma 62,99: $p = (0{,}867;\ 0{,}117;\ 0{,}016)$.
3. Bajar τ concentra la masa en el carácter más probable: las muestras son más correctas, pero más repetitivas. Subir τ (con $\tau = 2$, $p = (0{,}506;\ 0{,}307;\ 0{,}186)$) da más variedad y más errores.

Resultado. $\tau = 1$**:** $(0{,}665;\ 0{,}245;\ 0{,}090)$**;** $\tau = 0{,}5$**:** $(0{,}867;\ 0{,}117;\ 0{,}016)$**.** Bajar la temperatura hace las muestras más predecibles y repetitivas.

Comprobación. ambas distribuciones suman 1 y, cuando $\tau \to 0^+$, la distribución tiende a $(1, 0, 0)$.

Repasar: M15, §1.2, §1.3, §1.4, §2.3, §4.2, §4.4 y §5.2.

Apéndice B

Autoevaluación

B.1 Cálculo de la nota

La nota es la suma de los puntos de la parte A y de la parte B, sobre 100. El examen se aprueba con **70 puntos** (70 %). Aunque se apruebe, si en alguna parte o en algún módulo se obtiene menos del 50 % de los puntos posibles, debe repasarse ese contenido antes de continuar.

Cuadro B.1: Puntuación por parte.

Parte	Contenido	Módulos	Puntos	Obtenidos
A	Cuestiones (20 × 1 p)	M12–M15	20	
B.1	Bloque decoder	M14	15	
B.2	Atención con máscara causal	M13	15	
B.3	BPE a mano	M12	12	
B.4	Parámetros, FLOPs y memoria	M14	15	
B.5	RoPE y coste de la atención	M13, M14	11	
B.6	Entrenamiento del mini-GPT	M15	12	
Total			**100**	

Cuadro B.2: Puntuación por módulo. En el ejercicio B.5, el apartado (a) corresponde a M14 y los apartados (b) y (c), a M13.

Módulo	Elementos	Puntos	50 %	Obtenidos
M12. Tokenización	A.1–A.5; B.3	$5 + 12 = 17$	8,5	
M13. Atención	A.6–A.10; B.2; B.5(b,c)	$5 + 15 + 6 = 26$	13	
M14. Transformer	A.11–A.18; B.1; B.4; B.5(a)	$8 + 15 + 15 + 5 = 43$	21,5	
M15. Mini-GPT	A.19–A.20; B.6	$2 + 12 = 14$	7	
Total		**100**		

B.2 Puerta de salida de la fase III

Para pasar a la fase IV deben cumplirse las tres condiciones del método de estudio del curso:

1. en el test de cada módulo (M12, M13, M14 y M15), al menos 9 aciertos de 12;
2. en la autoevaluación del laboratorio interactivo de la fase, al menos un 70 %;
3. en este examen de hito, al menos 70 puntos de 100.

El proyecto de fase se considera superado con al menos 70 puntos en la rúbrica del cuadro C.1.

B.3 Qué hacer si no se alcanza el umbral

– Localizar en el cuadro B.3 las secciones correspondientes a cada fallo y anotar el error en el registro de errores, clasificado como de concepto, de cálculo o de despiste.

– Releer esas secciones y rehacer, sin consultar la solución, los ejercicios ⋆⋆ y ⋆⋆⋆ del tema correspondiente de los apuntes del módulo.

– Repetir el test del módulo afectado hasta alcanzar 9 de 12 y volver a resolver, al cabo de unos días, los ejercicios de la parte B en los que se perdió más de la mitad de los puntos.

– Si la nota está por debajo de 50, repasar la fase completa siguiendo el ciclo del método de estudio (teoría, laboratorio, práctica, ejercicios, test y tarjetas) antes de repetir el examen.

Cuadro B.3: Secciones que deben repasarse según los fallos.

Fallos	Repasar
A.1, A.4; B.3(b,c)	M12, §2.1 (entrenamiento de BPE) y §2.2 (codificación)
A.2, A.3; B.3(d)	M12, §1.3 (UTF-8), §1.5 (consecuencias para el español) y §2.4 (BPE a nivel de byte)
A.5	M12, §5.5 (secuencialidad y paso al Transformer)
A.6; B.2(a,b,d)	M13, §2.1 (forma matricial de la atención)
A.7; B.2(f)	M13, §2.3 (por qué se divide por $\sqrt{d_k}$)
A.8; B.2(c,e)	M13, §3.1 (máscaras aditivas) y §3.2 (causalidad)
A.9; B.5(b)	M13, §4.2 (parámetros de la atención multi-cabeza)
A.10; B.5(c)	M13, §4.3 (coste en operaciones) y §4.4 (memoria y contexto largo)
B.1(a–d)	M14, §1.2 (bloque decoder-only), §2.1 (flujo residual) y §3.1 (MLP posición a posición)
A.11, A.12; B.1(e)	M14, §2.2 (pre-norm frente a post-norm) y §2.3 (crecimiento del flujo)
A.13; B.4(a)	M14, §3.3 (SwiGLU)
A.14, A.15; B.5(a)	M14, §4.3 (RoPE)
A.16	M14, §1.4 (embeddings atados) y §5.2 (parámetros del modelo completo)
A.17, A.18; B.4(b–e)	M14, §5.1 a §5.3 (parámetros y memoria) y §6.2 y §6.3 (FLOPs y regla $6ND$)
A.19; B.6(a,b)	M15, §1.4 (pérdida de la predicción uniforme) y §2.3 (recuento de parámetros)
A.20; B.6(c,d)	M15, §1.3 (perplejidad), §4.2 (puntos de control) y §4.4 (lectura de curvas)
B.6(e)	M15, §5.2 (temperatura)

Siguiente paso. Superada la puerta de salida, se continúa con la fase IV, módulo 16: preentrenamiento a escala.

Bibliografía

[1] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Tarea 3, «Self-Attention and Transformers»: pregunta 1, «Attention exploration», y pregunta 2, «Position embeddings exploration». https://web.stanford.edu/class/cs224n/

[2] Equipo docente de CS336 (T. Hashimoto y P. Liang). CS336: Language Modeling from Scratch. Tarea 1, «Basics: Building a Transformer LM», versión 26.0.3. Stanford University, primavera de 2026. Se citan el §2.4 (ejemplo *bpe_example*), el §2.6.1 (ejemplo *bpe_encoding*), los §3.4 y §3.5 (problemas *rope*, *transformer_block*, *transformer_lm* y *transformer_accounting*), el §4.1 (perplejidad), el §4.3 (problema *adamw_accounting*), el §6 (problema *decoding*) y el §7.3 (problema *pre_norm_ablation*). https://github.com/stanford-cs336/assignment1-basics

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. En *Advances in Neural Information Processing Systems 35*, 2022. https://arxiv.org/abs/2205.14135

[4] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 2, «Words and Tokens» (§2.4.1 y ejercicio 2.3); cap. 7, «Transformers and Pretraining» (ejercicios 7.1, 7.2, 7.4 y 7.5). https://web.stanford.edu/~jurafsky/slp3/

[5] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[7] R. Sennrich, B. Haddow y A. Birch. Neural machine translation of rare words with subword units. En *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, págs. 1715–1725, 2016. https://aclanthology.org/P16-1162/

[8] N. Shazeer. GLU variants improve Transformer. Prepublicación, 2020. https://arxiv.org/abs/2002.05202

[9] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. *Neurocomputing*, 568, 2024. Prepublicación de 2021. https://arxiv.org/abs/2104.09864

[10] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30*, 2017. https://arxiv.org/abs/1706.03762

[12] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang y T.-Y. Liu. On layer normalization in the Transformer architecture. En *International Conference on Machine Learning (ICML)*, 2020. https://arxiv.org/abs/2002.04745